

이진트리(Binary Tree) 정리 (순회 및 용량 계산 응용 포함)

1. 트리(Tree) 개요

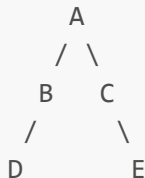
- **정의:** 계층적(계통적) 구조를 가지는 비선형 자료구조
 - **구성 요소:** 노드(Node)와 간선(Edge)
 - **특징:**
 - 하나의 루트 노드(root node) 존재
 - 순환(cycle)이 없음
 - 노드 간 관계는 부모-자식 구조
-

2. 이진트리(Binary Tree)

정의

- 모든 노드가 **최대 2개의 자식 노드**를 가지는 트리
- 자식 노드는 **왼쪽(left child)**, ****오른쪽(right child)****으로 구분
- 각 노드는 **0개, 1개, 2개의 자식 노드**를 가질 수 있음

예시



3. 이진트리의 종류

포화 이진트리 (Full Binary Tree)

- 모든 내부 노드가 **2개의 자식**을 가짐
- 모든 리프 노드는 **같은 레벨**에 존재

완전 이진트리 (Complete Binary Tree)

- 마지막 레벨을 제외한 모든 레벨이 노드로 가득 참
- 마지막 레벨은 왼쪽부터 노드가 채워짐

편향 이진트리 (Skewed Binary Tree)

- 모든 노드가 한쪽 자식만 가짐 (왼쪽 또는 오른쪽)
-

4. 구현 방식

배열 기반

- 인덱스 관계: $i \rightarrow 2i$ (왼쪽), $2i+1$ (오른쪽)

연결리스트 기반

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

5. 순회 방식

전위순회 (Preorder)

- 순서: 루트 → 왼쪽 → 오른쪽

```
void preorder(Node* node) {  
    if (!node) return;  
    cout << node->data << ' ';  
    preorder(node->left);  
    preorder(node->right);  
}
```

중위순회 (Inorder)

- 순서: 왼쪽 → 루트 → 오른쪽

```
void inorder(Node* node) {  
    if (!node) return;  
    inorder(node->left);  
    cout << node->data << ' ';  
    inorder(node->right);  
}
```

후위순회 (Postorder)

- 순서: 왼쪽 → 오른쪽 → 루트

```
void postorder(Node* node) {  
    if (!node) return;
```

```

    postorder(node->left);
    postorder(node->right);
    cout << node->data << ' ';
}

```

순회 예시 (트리)



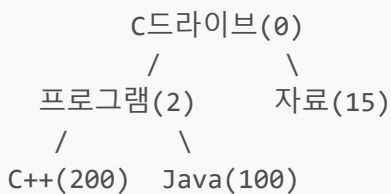
전위: A B D E C

중위: D B E A C

후위: D E B C A

6. 응용: 디렉토리 구조 용량 계산

트리 모델 예시



후위순회를 통해 계산:

- C++ + Java = 300
- 프로그램 = 300 + 2 = 302
- 자료 = 15
- C드라이브 전체 = 302 + 15 = 317

확장 예시: D드라이브

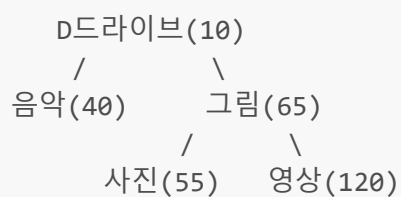


그림 폴더 = 55 + 120 + 65 = 240

D드라이브 전체 = 10 + 40 + 240 = 290

전체 용량

- C드라이브: 317MB
- D드라이브: 290MB
- 전체: $317 + 290 = \mathbf{607MB}$

7. 요약 표

구분	정의	최대 노드 수	특징
포화	모든 레벨이 가득 참	$(2^h - 1)$	완벽한 구조
완전	마지막 레벨 왼쪽부터 채움	$(\leq 2^h - 1)$	배열 표현 유리
편향	한쪽 자식만 존재	h	메모리 낭비 가능

8. 활용 분야

- 컴파일러(AST)
- 수식 파싱 및 계산기
- 디렉토리 용량 계산
- 탐색/정렬 알고리즘

실습을 통해 직접 이진트리 구조를 그려보고, 후위순회 방식으로 용량을 계산하는 과정은 자료구조의 이해도를 높이는 중요한 연습입니다.